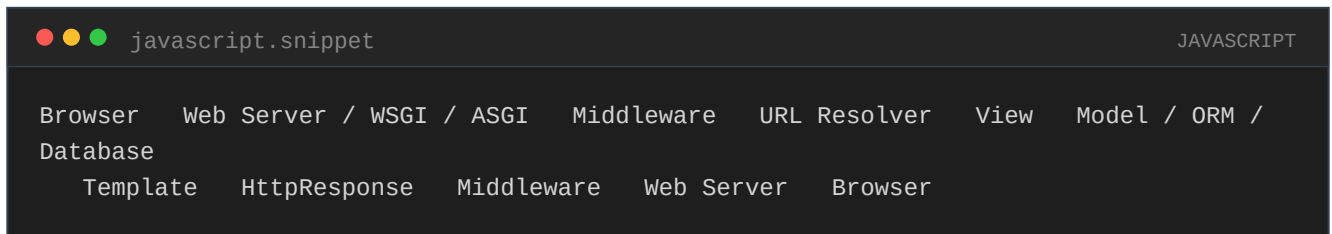


Django Request Lifecycle -- Step by Step

The **Django Request Lifecycle** describes the complete process from the moment a user sends an HTTP request from a browser until Django sends the HTTP response back to the browser.

Complete Flow



1. User Request -- Browser Server

Suppose the user enters:

```
https://example.com/products/5/
```

The browser creates an HTTP request and sends it to the server.

An HTTP request may contain:

```
HTTP MethodURL / PathHeadersCookiesQuery ParametersRequest Body / Form Data
```

Example:

```
GET /products/5/ HTTP/1.: example.comCookie: sessionid=abc123
```

Here:

```
GET HTTP method/products/5/ URL pathCookie Browser cookie
```

2. Web Server WSGI / ASGI

The incoming request reaches the Django application through a server/application interface.

Django supports two major interfaces:

```
javascript.snippet JAVASCRIPT
WSGIASGI
```

WSGI

WSGI = Web Server Gateway Interface

It is traditionally used for synchronous Python web applications.

Examples of servers that can run Django through WSGI:

```
javascript.snippet JAVASCRIPT
GunicornuWSGI
```

ASGI

ASGI = Asynchronous Server Gateway Interface

ASGI provides support for asynchronous applications and protocols such as WebSockets.

Examples:

```
javascript.snippet JAVASCRIPT
DaphneUvicornHypercorn
```

Important

```
javascript.snippet JAVASCRIPT
WSGI Traditional synchronous interfaceASGI Async-capable interface
```

Using ASGI does not automatically make all Django code asynchronous.

3. Django Middleware -- Request Phase

After entering Django, the request goes through the **middleware chain**.

Middleware is a layer that can process both requests and responses.

Common Django middleware includes:

```
javascript.snippet JAVASCRIPT
SecurityMiddlewareSessionMiddlewareCommonMiddlewareCsrfViewMiddlewareAuthenticationMiddleware
```

Basic flow:

```
javascript.snippet JAVASCRIPT
Request Middleware 1 Middleware 2 Middleware 3 URL Resolver
```

Middleware can:

- Modify the request
- Block a request
- Perform security checks
- Load session information
- Add authentication-related information
- Check CSRF
- Modify the response
- Add response headers
- Set cookies

4. URL Resolver -- `urls.py`

After the request passes through the relevant middleware processing, Django resolves the requested URL.

Example:

```
javascript.snippet JAVASCRIPT

from django.urls import path
from .views import product_detail
urlpatterns = [
    path("products/", product_detail),]
```

If the browser requests:

```
javascript.snippet JAVASCRIPT

/products/5/
```

Django matches it with:

```
javascript.snippet JAVASCRIPT

products/
```

Therefore:

```
javascript.snippet JAVASCRIPT

id = 5
```

Django then determines which **view callable** should handle the request.

If the URL Does Not Match

If no URL pattern matches the request:

```
javascript.snippet JAVASCRIPT

404 Not Found
```

is returned.

5. View Execution

Once the URL resolver determines the view, Django executes it.

Example:

```
javascript.snippet JAVASCRIPT

from django.shortcuts import renderdef product_detail(request, id):    product =
Product
.objects.get(id=id)    return render(        request,        "product.html",        {"
product
": product}    )
```

The view can:

```
javascript.snippet JAVASCRIPT

Receive the request        Execute application logic        Query the database        Process
data        Create/return a response
```

A view can be:

Function-Based View

```
javascript.snippet JAVASCRIPT

def product_detail(request, id):    ...
```

Class-Based View

```
javascript.snippet JAVASCRIPT

class ProductDetailView(DetailView):    ...
```

6. Model ORM Database

If the view needs data from the database, Django's ORM can be used.

Example:

```
javascript.snippet JAVASCRIPT

product = Product.objects.get(id=id)
```

The general flow is:

```
javascript.snippet JAVASCRIPT
View Django ORM Database Query Result View
```

The Django ORM allows developers to work with database data using Python objects and methods.

For example:

```
javascript.snippet JAVASCRIPT
Product.objects.get(id=5)
```

results in a database query being executed.

7. Template Rendering

If the response needs to be HTML, Django can use its template system.

Example:

```
javascript.snippet JAVASCRIPT
return render(request, "product.html", {"product": product})
```

The process is:

```
javascript.snippet JAVASCRIPT
Template    +Context Data    Django Template Engine    Rendered HTML
```

Template:

```
javascript.snippet JAVASCRIPT
{{ product.name }}{{ product.price }}
```

Context:

```
javascript.snippet JAVASCRIPT
{    "product": product}
```

Django replaces the template variables with the corresponding data and produces the final HTML.

8. HTTP Response Is Created

A view ultimately returns an HTTP response.

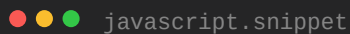
Common Django response classes include:

A header for a code snippet in a dark-themed editor, showing three colored circles (red, yellow, green) and the text "javascript.snippet".

JAVASCRIPT

```
HttpResponse JsonResponse HttpResponseRedirect StreamingHttpResponse TemplateResponse
```

HttpResponse

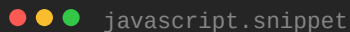
A header for a code snippet in a dark-themed editor, showing three colored circles (red, yellow, green) and the text "javascript.snippet".

JAVASCRIPT

```
from django.http import HttpResponseRedirectdef hello(request):    return HttpResponseRedirect("Hello")
```

JsonResponse

Useful for APIs:

A header for a code snippet in a dark-themed editor, showing three colored circles (red, yellow, green) and the text "javascript.snippet".

JAVASCRIPT

```
from django.http import JsonResponsedef api(request):    return JsonResponse({    "name": "Faruk",    "age": 25    })
```

Redirect

A header for a code snippet in a dark-themed editor, showing three colored circles (red, yellow, green) and the text "javascript.snippet".

JAVASCRIPT

```
from django.shortcuts import redirectdef my_view(request):    return redirect("home")
```

Important

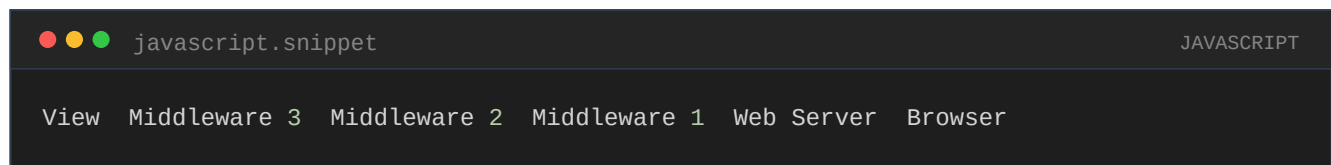
`render ( )` does not directly send HTML to the browser.

It renders the template and returns a response object, which the view returns to Django.

9. Middleware -- Response Phase

After the view returns a response, the response travels back through the middleware chain.

The direction is effectively reversed:

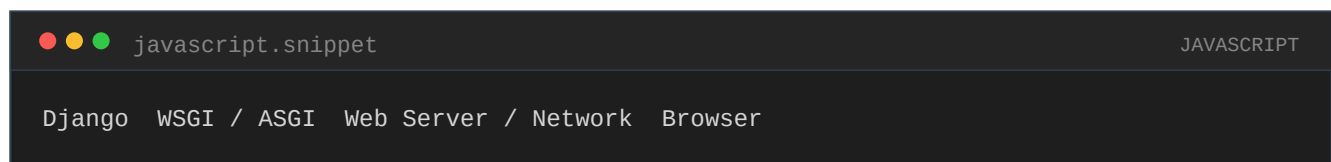


Middleware can perform operations such as:

- Adding response headers
- Setting cookies
- Modifying the response
- Security-related processing
- Caching-related processing

10. Web Server Browser

After Django finishes processing the response, it is sent back through the server/interface layer to the browser.



The browser receives the HTTP response and displays the result to the user.

Complete Request Lifecycle Diagram



Real Example

Suppose the user requests:

```
javascript.snippet
JAVASCRIPT

GET /products/5/
```

Step 1 -- Browser

The browser sends:

```
javascript.snippet
JAVASCRIPT

GET /products/5/
```

Step 2 -- WSGI / ASGI

The request reaches the Django application.

Step 3 -- Middleware

Middleware processes the request.

For example:

```
javascript.snippet
JAVASCRIPT

SessionAuthenticationCSRFSecurity
```

Step 4 -- URL Resolver

Django matches:

```
javascript.snippet JAVASCRIPT
path("products/", product_detail)
```

So:

```
javascript.snippet JAVASCRIPT
id = 5
```

Step 5 -- View

Django executes:

```
javascript.snippet JAVASCRIPT
def product_detail(request, id):
```

Step 6 -- ORM / Database

The view executes:

```
javascript.snippet JAVASCRIPT
Product.objects.get(id=5)
```

The product is retrieved from the database.

Step 7 -- Template

The view renders:

```
javascript.snippet JAVASCRIPT
render(request, "product.html", {"product": product})
```

Step 8 -- Response

Django creates the HTTP response.

Step 9 -- Middleware

The response passes back through the middleware chain.

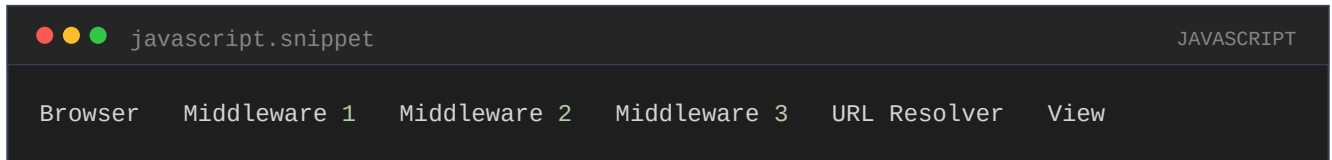
Step 10 -- Browser

The browser receives the final response and displays the page.

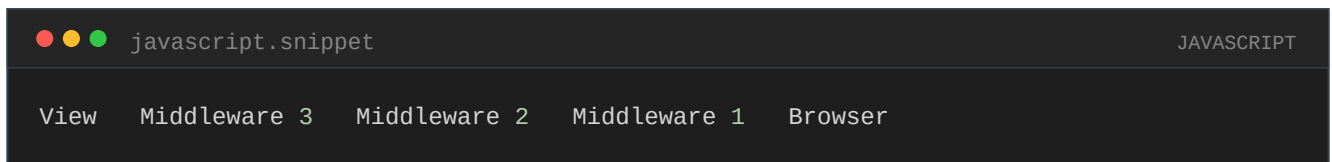
Request vs Response Flow

This is very important for interviews.

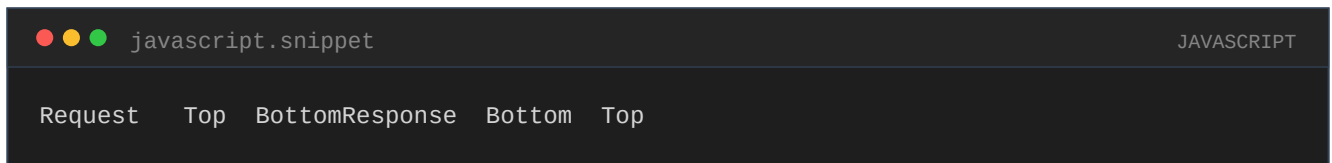
Request Flow



Response Flow



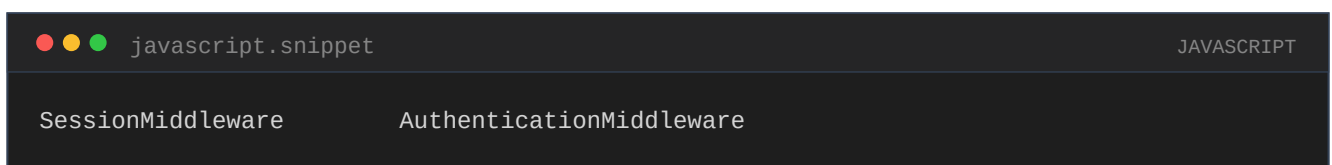
Shortcut



Why Does Middleware Order Matter?

Middleware order is important because middleware can depend on other middleware and because the processing order affects application behavior.

For example:



AuthenticationMiddleware relies on session information, so **SessionMiddleware** should come before it.

Therefore:

```
javascript.snippet JAVASCRIPT

Middleware order is not random.
```

The order can affect how requests and responses are processed.

Important Concepts

1. URL Resolver

```
javascript.snippet JAVASCRIPT

URL Which View should execute?
```

The URL resolver matches the requested URL with URL patterns.

2. View

```
javascript.snippet JAVASCRIPT

Request   Application Logic   Response
```

The view is a central part of request processing.

3. Model / ORM

```
javascript.snippet JAVASCRIPT

Python Code   Django ORM   Database
```

The ORM simplifies database interaction.

4. Template

```
javascript.snippet JAVASCRIPT

Template + Context   HTML
```

Templates are commonly used to generate HTML responses.

5. Middleware

● ● ● javascript.snippet

JAVASCRIPT

Request / Response Middleware Processing

Middleware provides reusable processing around views.

6. WSGI / ASGI

● ● ● javascript.snippet

JAVASCRIPT

Django Server [Interface](#)

● ● ● javascript.snippet

JAVASCRIPT

WSGI Traditional synchronous interface ASGI [Async-capable interface](#)

Interview Questions

Q1. What is the Django Request Lifecycle?

The Django Request Lifecycle is the complete process through which an HTTP request travels from the browser to Django, is processed by middleware, URL routing, views, ORM/database and other components, and finally returns an HTTP response to the browser.

Q2. What does the URL Resolver do?

The URL resolver matches the requested URL against URL patterns in `urls.py` and determines which view should handle the request.

Q3. What is Middleware?

Middleware is a reusable layer that processes HTTP requests and responses before and

after the view executes.

Q4. Can middleware block a request?

Yes. Middleware can return a response directly without allowing the request to reach the view.

Q5. What does Django ORM do?

Django ORM provides a Python-based interface for interacting with the database and constructing/executing database queries.

Q6. What does `render()` do?

`render()` combines a template with context data and returns an HTTP response containing the rendered content.

Q7. What is the difference between WSGI and ASGI?

```
javascript.snippet JAVASCRIPT

WSGI Traditional synchronous Python web interface
ASGI Async-capable Python web interface
```

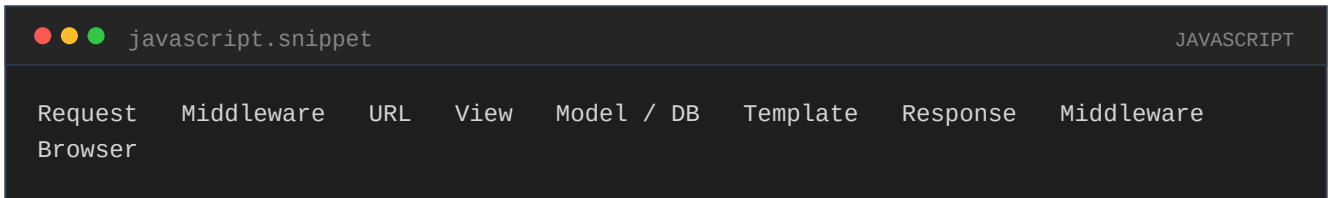
ASGI also supports protocols such as WebSockets.

Quick Revision

```
javascript.snippet JAVASCRIPT

Browser  WSGI / ASGI  Middleware  URL Resolver  View  ORM / Database  Template
Response
Middleware  Browser
```

Easy Memory Trick



Five Most Important Points

```
1. Middleware order matters.2. URL Resolver determines which view handles the URL.3. View handles the main request-processing logic.4. Django ORM handles database interaction.5. Response processing travels back through middleware.
```